

Procesamiento de Lenguaje Natural III

Docentes:

Mg. Oksana Bokhonok - FIUBA

Esp. Abraham Rodriguez - FIUBA

Programa de la materia

1. RAG avanzado y personalización de soluciones.
2. Seguridad. Ética y alineación de modelos con valores humanos.
- 3. Métricas, Optimización y Herramientas para la construcción de Agentes.**

La diferencia entre los KPI

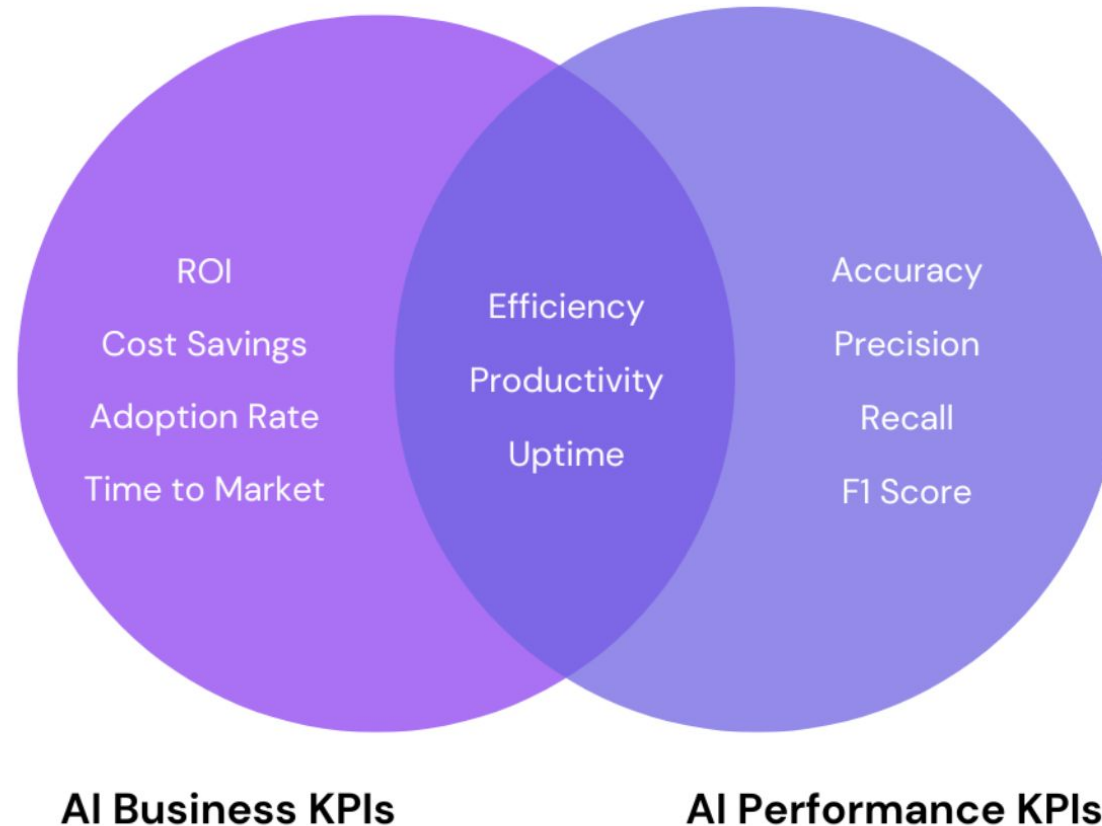
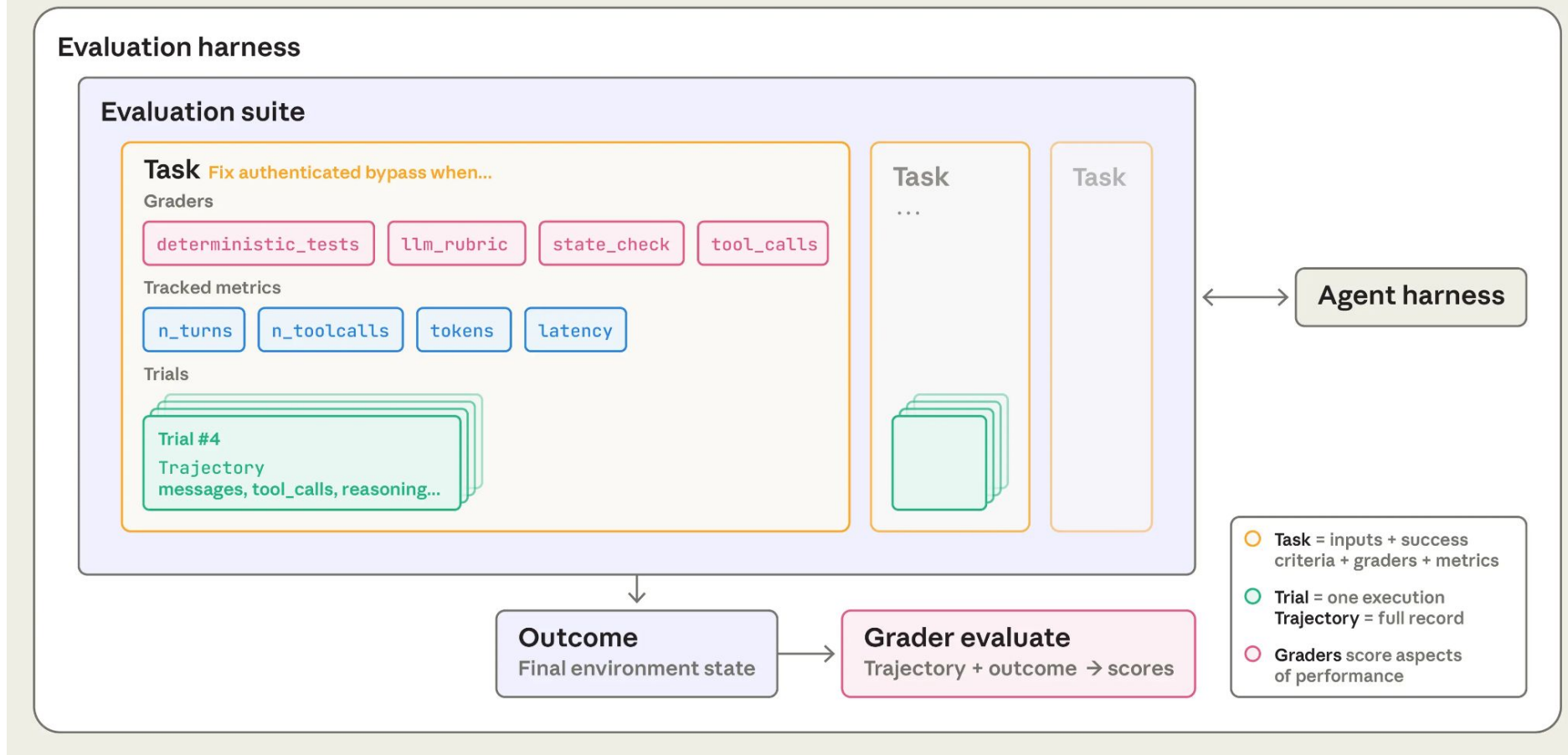


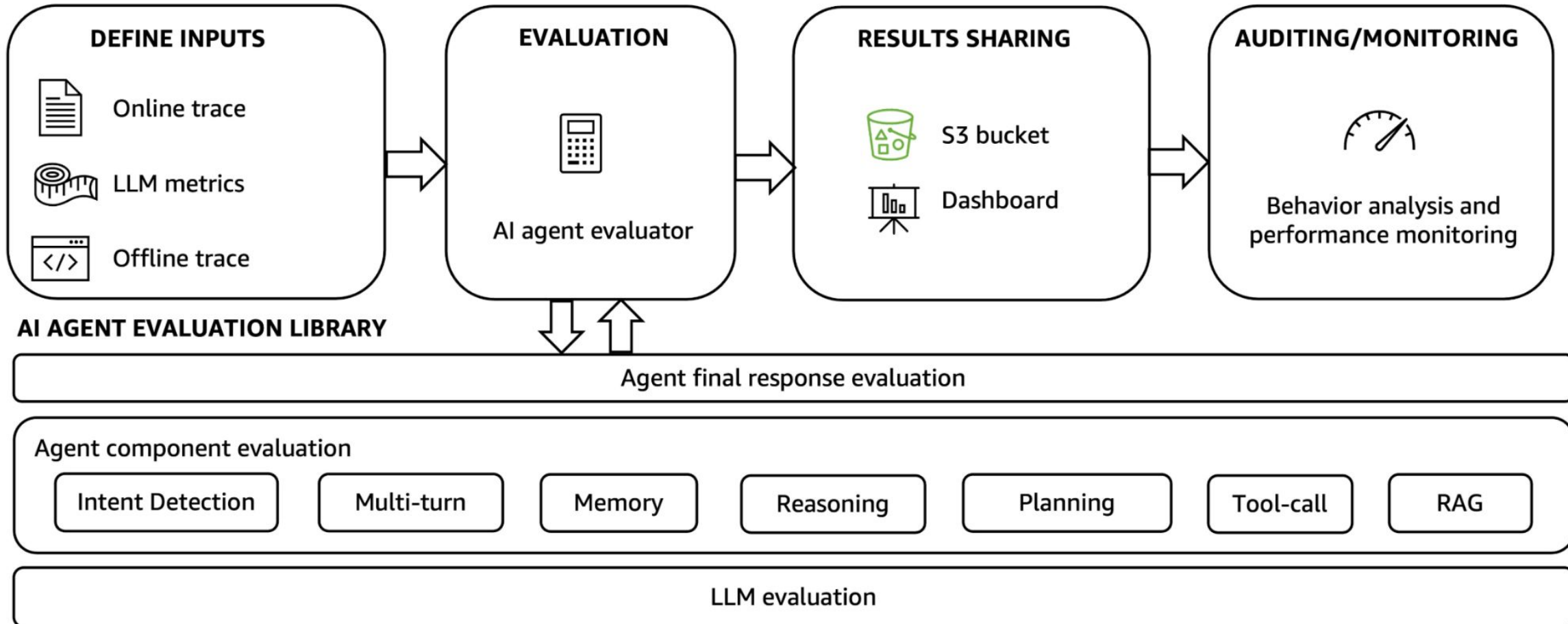
Fig. 1. Comparación de los KPI de negocio de la IA y los KPI de rendimiento de la IA.

[KPIs de rendimiento de IA: Medir el éxito | Ultralytics](#)

Components of Evaluations for Agents



Etapas de evaluación



1 — Retrieval incluye re-rank y fusión sparse+denso

★ Recall@k	¿El pasaje correcto aparece en el top-k? Meta: ≥ 0.85 @20. La métrica de entrada — si falla aquí, todo lo demás falla.
★ nDCG@k	¿Lo relevante queda arriba? Penaliza cuando el soporte correcto aparece en posición 18 en lugar de 2.
· Hit rate@k	% de queries con al menos 1 pasaje válido en top-k. Versión binaria de Recall@k — útil para reportes ejecutivos.
· nDCG lift (re-ranker)	Mejora del re-ranker sobre el candidato inicial.
· RRF / hybrid gain	Mejora de sparse+denso vs cada señal sola.
· Context Precision	% del contexto incluido que realmente respalda la respuesta. — es una métrica de calidad del retriever.
· Compresión efectiva	Tokens de contexto vs soporte real. Menos "paja" con igual recall.
· Freshness	Lag medio entre cambio en fuente e índice actualizado. Importante solo si el dominio cambia frecuentemente.

MRR se puede omitir si ya usás nDCG — miden cosas muy similares. Diversidad/De-dup es útil en retrieval de documentos largos, secundaria en la mayoría de casos.

Evaluación - Retrieval

Evaluar qué tan bien el sistema recupera la información correcta antes de generar una respuesta. Porcentaje de queries donde el documento o pasaje correcto aparece dentro del top-k recuperado.

Recall@k

Recall@20 = 0.85 → 85% de las consultas encontraron evidencia correcta en top-20

Hit Rate@k

Mide si existe al menos un resultado válido dentro del top-k. Más simple que Recall.

nDCG@k

Evalúa calidad del ranking. No solo importa recuperar: importa qué tan arriba aparece.

Penaliza: documentos relevantes demasiado abajo.



Recall@k

$$\text{Recall@K} = \frac{\text{true_positives@K}}{\text{true_positives@K} + \text{false_negatives@K}}$$



Hit Rate@k

$$\text{HitRate@K} = \begin{cases} 1, & \text{if at least one hit in retrieved set} \\ 0, & \text{if no hits} \end{cases}$$



nDCG@k

$$\text{nDCG}_p = \frac{\text{DCG}_p}{\text{IDCG}_p}$$

$$\text{DCG}_p = \sum_{i=1}^p \frac{2^{\text{rel}_i - 1}}{\log_2(i+1)}$$

$$\text{IDCG}_p = \sum_{i=1}^p \frac{2^{\text{rel}_i^{\text{ideal}} - 1}}{\log_2(i+1)}$$

- p : rank position cutoff
- rel_i : relevance score of the document at rank i
- $\text{rel}_i^{\text{ideal}}$: relevance of document at rank i in ideal ordering

Evaluación - Retrieval

MRR (Mean Reciprocal Rank)



Evalúa: qué tan rápido aparece el primer documento correcto. Ideal para: QA agentes con poco contexto y baja tolerancia a latencia

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}$$

Calidad del retrieval: Diversidad / De-dup



Evitar: chunks redundantes múltiples variantes del mismo contenido.
Problema típico: Top-5 = mismo documento repetido.

Freshness



Mide desactualización entre: fuente original e índice vectorial. Crítico en: noticias, noticias, finanzas, monitoreo, documentación viva.

Age-Weighted Recall

Solving Freshness in RAG: A Simple Recency Prior and the Limits of Heuristic Trend Detection

2 — Generación incluye métricas de contexto que dependen del output

★ Faithfulness	% de afirmaciones respaldadas por el contexto y no contradichas. unifica "hallucination rate" — son la misma cosa en positivo/negativo.
★ Context Recall (Coverage)	¿El soporte necesario está en el contexto final y fue usado? Detecta si el modelo ignoró evidencia relevante que sí estaba disponible.
★ Abstención correcta	Cuando no hay evidencia, ¿el agente dice "no sé"? Crítico para confiabilidad — los falsos positivos de confianza son el peor fallo visible.
Exactitud EM / F1	Sobre conjunto con ground truth. Requiere datos anotados — costoso de mantener, pero la única métrica objetiva de calidad de respuesta.
Citación correcta	% de respuestas que citan los pasajes adecuados (no solo cualquier doc).
Formato / contrato	% de JSON/SQL válidos, schemas, plantillas. Muy importante en agentes que entregan outputs a sistemas downstream.

Métricas y fórmulas de Contexto



Faithfulness

Afirmaciones soportadas
por contexto

Total de afirmaciones
generadas



Context Recall

Claims de referencia
soportados por contexto

Total de claims de
referencia



Abstención correcta

Casos sin evidencia con
abstención

Total de casos sin
evidencia

3 — Robustez & Seguridad métricas de estrés, no de operación normal

★ Caída ante paráfrasis / ruido	Delta de Recall@k y Faithfulness ante reformulaciones, typos, queries largas. El indicador más realista de comportamiento en producción real.
· OOD / Domain shift	Desempeño con temas fuera del dominio de entrenamiento/indexación. Útil antes de lanzar a nuevos dominios.
· Políticas (PII, jailbreak)	Filtrado de datos sensibles, cumplimiento normativo, resistencia básica a ataques de prompt. Obligatorio en producción, no negociable.

Métricas de Robustez



Robustness Drop

$$\text{Score_original} - \text{Score_perturbado}$$



Robustness Drop

$$\alpha \times (\text{Recall@k_original} - \text{Recall@k_perturbado}) + \beta \times (\text{Faithfulness_original} - \text{Faithfulness_perturbado})$$



Con pesos iguales

$$\text{Drop} = 0.5 \times \Delta\text{Recall@k} + 0.5 \times \Delta\text{Faithfulness}$$

Score_original = desempeño con la pregunta normal.

Score_perturbado = desempeño con la pregunta reformulada, con typos, ruido o query más larga.

α y β = pesos según importancia de cada métrica.

4 — Operación (SLOs) lo que se monitorea en producción en tiempo real

- ★ **Latencia end-to-end (p50/p95)** El único número que el usuario final siente directamente.
- ★ **Costo por respuesta (tokens / \$)** Tokens de entrada + salida × precio del modelo + costo de herramientas. La palanca más directa de optimización económica.
- **Tasa de errores** Timeouts, context overflow, fallos de fetch, errores de formato. Separar por tipo — cada uno tiene causa raíz distinta.
- **Uso de paralelismo** % de tareas ejecutadas en paralelo vs secuen

Latencia del retriever (p50/p95) es una sub-métrica de latencia total

Métricas Operativas

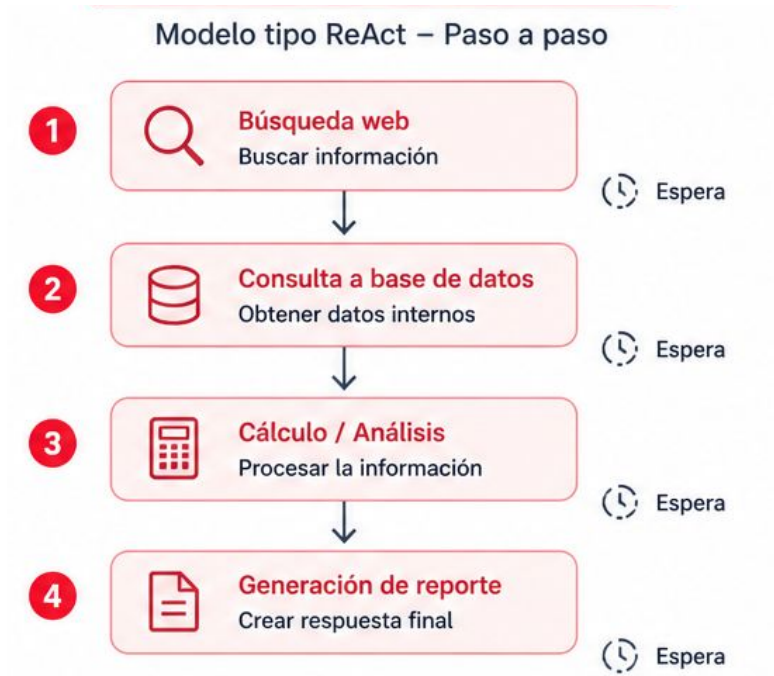
`latencia_e2e = timestamp_respuesta_final -
timestamp_request_usuario`

`p50 = percentil 50 de todas las latencias_e2e`

`p95 = percentil 95 de todas las latencias_e2e`

`costo_respuesta = costo_tokens_entrada +
costo_tokens_salida + costo_herramientas +
costo_infra`

Agentes optimizados - paralelización



fan-out / fan-in

LAS 3 REGLAS DEL SCHEDULER INTELIGENTE

1 Detectar independencia antes de ejecutar

El LLM genera el grafo de tareas. El scheduler analiza dependencias: si A no necesita el resultado de B, corren juntas.
Herramienta: `asyncio.gather()`, LangGraph con nodos paralelos.

2 No bloquear en I/O — todo async

Las llamadas a APIs, bases de datos y herramientas externas son I/O puro. Usar `httpx async`, `drivers async` (`asyncpg`, `motor`). Un agente sync que llama 4 APIs tarda 4x más sin ningún beneficio.

3 El LLM es el recurso más caro — usarlo lo menos posible

Clasificaciones simples, extracción de campos, cálculos → modelo liviano o motor determinístico. El LLM grande solo entra para planificación y síntesis final. El scheduler decide qué modelo lanzar para cada nodo del grafo.

```
result_web, result_db, result_docs, result_api = await asyncio.gather(
    buscar_web_async(),
    consultar_db_async(),
    buscar_documentos_async(),
    llamar_api_async()
)
```

Paralelizacion y State

- **Usar claves de estado anotadas:** usá siempre claves de estado anotadas con reducers adecuados para los campos que van a ser actualizados por nodos en paralelo.
- **Mantener tipos de retorno consistentes:** asegurate de que los nodos paralelos devuelvan la misma estructura de datos para los campos que actualizan.
- **Minimizar dependencias:** diseñá los nodos paralelos para que sean lo más independientes posible, de modo de aprovechar mejor la ejecución concurrente.
- **Manejar errores de forma robusta:** considera cómo los errores en una rama paralela deberían afectar la ejecución general. En algunos casos, puede convenir continuar aunque una rama falle.
- **Reducers personalizados:** implementa reducers personalizados cuando necesites fusionar estados complejos que van más allá de una simple concatenación de listas.

Clave de estado y reducer

Una **clave de estado anotada** es un campo del estado que no solo tiene un tipo, sino también una regla de combinación. Y Un **reducer** es una función que define cómo combinar el valor anterior del estado con el nuevo valor que devuelve un nodo.

Nodo A devuelve:

```
{"messages": ["Respuesta del agente A"]}
```

Nodo B devuelve:

```
{"messages": ["Respuesta del agente B"]}
```

Con reducer operator.add, el resultado final es:

```
{  
  "messages": [  
    "Respuesta del agente A",  
    "Respuesta del agente B"  
  ]  
}
```

TAREA	EJEMPLOS DE MODELOS	COSTO
Planificación compleja	GPT-4o Claude Sonnet 4 Claude Opus 4 Gemini 2.5 Pro Qwen3-72B <i>Necesita razonamiento multi-paso y tool use robusto</i>	... Alto
Extracción / Estructuración	GPT-4o mini Claude Haiku 3.5 Gemini 2.0 Flash Qwen3-8B Llama 3.1 8B <i>Ideal con structured outputs / JSON mode</i>	.. Medio
Clasificación simple	GPT-4o mini Gemini 2.0 Flash Lite Llama 3.2 3B Qwen3-1.7B fine-tuned BERT / DistilBERT <i>Para alto volumen, un modelo fine-tuneado gana en costo</i>	. Bajo
Búsqueda / Embeddings	text-embedding-3-small text-embedding-3-large text-embedding-004 nomic-embed / BGE-M3 <i>BGE-M3 y nomic-embed son open source y muy competitivos</i>	. Bajo
Cálculo / Lógica	Python (sympy, numexpr) Wolfram Alpha API o4-mini (razonamiento) Gemini 2.5 Flash (thinking) <i>Preferir motor determinístico; LLM solo para parsear la consulta</i>	. Muy bajo
Verificación / Factualidad	Claude Sonnet 4 Gemini 2.5 Pro GPT-4o + web search RAG + modelo mediano <i>Recomendado: RAG con retrieval real, no LLM solo</i>	.. Medio / alto

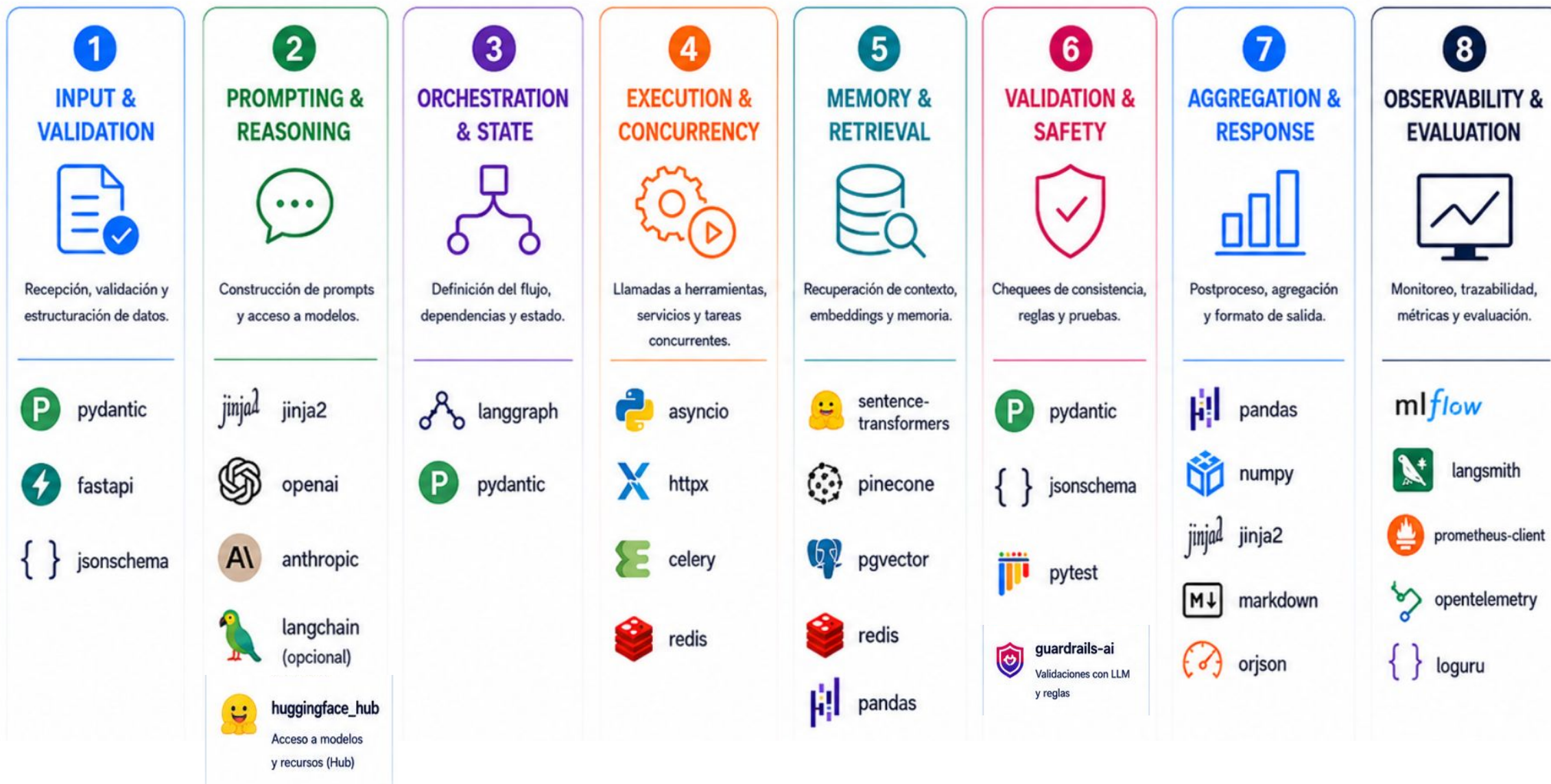
UBA
fiuba
FACULTAD DE INGENIERÍA



Agentes optimizados - uso de modelos

Ecosistema python

en cada etapa constructiva de un Agente



Ecosistema python - INPUT/VALIDATION

	Qué es	Ejemplo básico	Qué puede hacer	Fuente oficial
 pydantic	Validación y tipado de datos usando modelos Python	<pre>from pydantic import BaseModel</pre>	Validar inputs, salidas estructuradas , serializar JSON, generar schemas, manejar estado de agentes	GitHub - pydantic/pydantic: Data validation using Python type hints · GitHub / Pydantic: Simplifying Data Validation in Python – Real Python / pydantic · PyPI
 fastapi	Framework para construir APIs y validación de entrada	<pre>from fastapi import FastAPI</pre>	Crear APIs REST, validar requests automáticamente, manejar endpoints, integración con Pydantic	https://fastapi.tiangolo.com , FastAPI · PyPI
 jsonschema	Validación de estructuras JSON usando schemas	<pre>import jsonschema</pre>	Validar JSON, contratos de API, verificar estructura, reglas obligatorias	https://python-jsonschema.readthedocs.io , jsonschema · PyPI

Ecosistema python - Pydantic

```
from pydantic import BaseModel
```

```
class User(BaseModel):  
    name: str  
    age: int
```

→ schema en un .py separado

```
user = User(name="Juan", age="30")
```

Qué pasa acá:

- toma "30" (string)
- lo convierte a int (si no logra convertir (ejemplo: "abc") → lanza un error)
- valida que cumple el schema
- crea un objeto seguro

Ecosistema python - prompting/reasoning



jinja2

Motor de plantillas para generar texto dinámico

```
from jinja2 import Template
```

Templates de prompts, generación de HTML, formateo de respuestas, variables dinámicas

jinja.palletsprojects.com



openai

Cliente para acceso a modelos OpenAI

```
from openai import OpenAI
```

Chat completions, embeddings, structured outputs, tool calling

platform.openai.com/docs



anthropic

Cliente para acceso a modelos Claude

```
from anthropic import Anthropic
```

Generación de texto, reasoning, tool use, manejo de contexto largo

docs.anthropic.com



langchain
(opcional)

Framework opcional de orquestación para LLMs y agentes

```
from langchain_openai import ChatOpenAI
```

Chains, prompts, tools, agentes, memoria, RAG

python.langchain.com



huggingface_hub

Acceso a modelos y recursos (Hub)

Cliente para acceso a modelos y recursos de Hugging Face

```
from huggingface_hub import snapshot_download
```

Descargar modelos, datasets, embeddings y repositorios

huggingface.co/docs/huggingface_hub

Ecosistema python - orquestación/state



langgraph

Grafos de ejecución
con estado

Framework para
construir agentes con
grafos de ejecución y
estado

```
from langgraph.graph  
import StateGraph
```

Orquestación, flujos
condicionales, paralelización,
manejo de estado, multi-agente

<https://docs.langchain.com/langgraph>



pydantic

Modelos de estado
y datos

Librería de validación y
tipado de datos

```
from pydantic import  
BaseModel
```

Modelos de estado, validación
de datos, structured outputs,
schemas

<https://docs.pydantic.dev>

Ecosistema python - tareas y herramientas



Librería estándar de Python
para programación asíncrona

```
import  
asyncio
```

Concurrencia, ejecución paralela
I/O-bound, coroutines, task
scheduling

<https://docs.python.org/3/library/asyncio.html>



Cliente HTTP moderno
síncrono y asíncrono

```
import httpx
```

Requests async, APIs REST,
streaming, autenticación, HTTP/2

<https://www.python-httpx.org>



Sistema distribuido de colas y
ejecución de tareas

```
from celery  
import Celery
```

Workers distribuidos, background
jobs, retries, scheduling, pipelines

<https://docs.celeryq.dev>



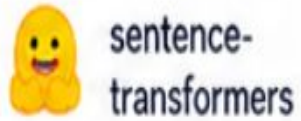
Base en memoria usada
como broker, cache y
mensajería

```
import redis
```

Backend para colas, pub/sub, cache,
estado compartido, rate limiting

<https://redis.io/docs/latest/develop/client/redis-py>

Ecosistema python - memoria y retrieval



Librería para generar embeddings semánticos de texto

```
from sentence_transformers import SentenceTransformer
```

Embeddings, búsqueda semántica, similitud, clustering, retrieval

www.sbert.net



Base vectorial gestionada para almacenar y buscar embeddings

```
from pinecone import Pinecone
```

Vector DB, búsqueda por similitud, retrieval, RAG

docs.pinecone.io



Extensión de PostgreSQL para almacenamiento y búsqueda vectorial

```
CREATE EXTENSION vector;
```

Búsqueda vectorial en PostgreSQL, similitud, retrieval, RAG

github.com/pgvector/pgvector



Base en memoria para cache y almacenamiento rápido

```
import redis
```

Cache, estado compartido, sesiones, pub/sub, almacenamiento temporal

redis.io/docs/latest/develop/clients/redis-py/

Ecosistema python - seguridad y reglas



pydantic

Librería de validación y
tipado fuerte de datos

```
from pydantic  
import BaseModel
```

Validar datos, structured outputs,
modelos de estado, schemas

<https://docs.pydantic.dev>



jsonschema

Librería para validación de
estructuras JSON

```
import jsonschema
```

Validar esquemas, contratos de API,
estructuras obligatorias

<https://python-jsonschema.readthedocs.io>



pytest

Framework de testing para
Python

```
import pytest
```

Tests unitarios, integración,
validaciones automáticas, regresión

<https://docs.pytest.org>



guardrails-ai

Validaciones con LLM
y reglas

Framework para validación y
control de outputs de LLMs

```
from guardrails  
import Guard
```

Validaciones con reglas, structured
outputs, seguridad, control de
respuestas

<https://www.guardrailsai.com/docs>

Ecosistema python - agregación y respuestas



markdown

Formateo de
salida

Librería para convertir Markdown a
HTML/texto renderizable

```
import markdown
```

Formateo de salida, documentación, render
de respuestas estructuradas

<https://python-markdown.github.io>



orjson

Serialización JSON
alta performance

Librería de serialización JSON de
alta performance

```
import orjson
```

Serialización rápida, respuestas JSON
eficientes, APIs de alto rendimiento

<https://github.com/ijl/orjson>

Ecosistema python - observabilidad y métricas



Plataforma para tracking y gestión del ciclo de vida de modelos ML

```
import mlflow
```

Tracking de experimentos, métricas, modelos, versiones y artifacts

[MLflow - Open Source AI Platform for Agents, LLMs & Models](https://mlflow.org/docs/latest/index.html)

[mlflow.langchain](https://mlflow.org/docs/latest/langchain.html)



langsmith

Plataforma de observabilidad para agentes y aplicaciones LLM

```
from langsmith import Client
```

Trazas de agentes, debugging, evaluación y monitoreo de pipelines LLM

<https://docs.smith.langchain.com>



prometheus-client

Cliente Python para métricas compatibles con Prometheus

```
from prometheus_client import Counter
```

Métricas, monitoreo, contadores, latencia y exposición de endpoints

https://github.com/prometheus/client_python



opentelemetry

API estándar para tracing y observabilidad distribuida

```
from opentelemetry import trace
```

Trazas distribuidas, métricas, observabilidad y monitoreo estándar

<https://opentelemetry.io/docs/languages/python>



loguru

Librería de logging simple y avanzada para Python

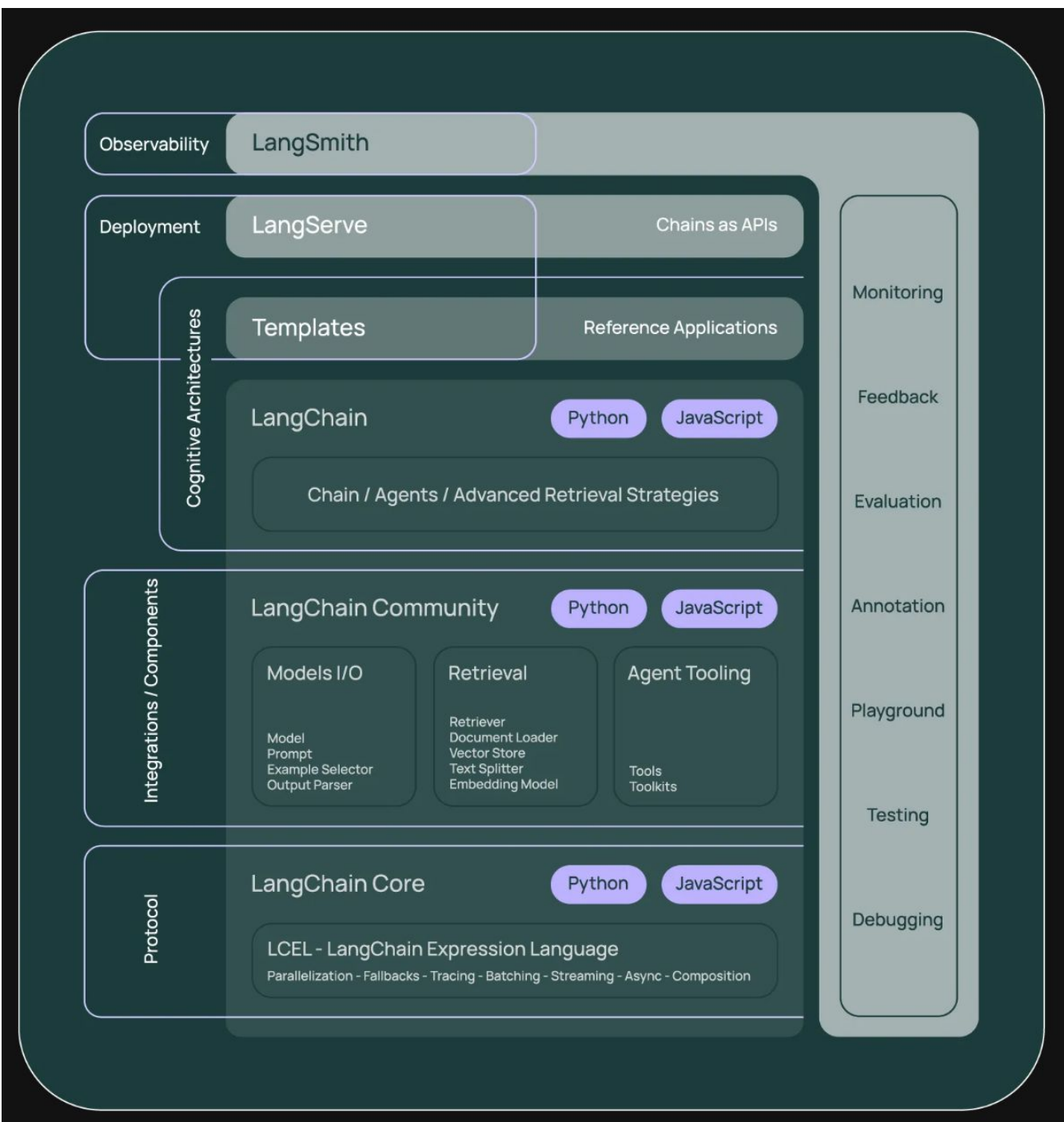
```
from loguru import logger
```

Logging estructurado, manejo de errores, rotación de logs y debugging

<https://loguru.readthedocs.io>

Familia Lang

LangChain	Framework para construir aplicaciones con LLMs: prompts, modelos, tools, retrievers, parsers e integraciones.	Cuando necesitás conectar modelos con datos, herramientas externas, RAG o flujos simples de aplicación.	LangChain docs: python.langchain.com LangChain Academy: github.com/langchain-ai/langchain-academy
LangGraph	Framework para construir agentes como grafos con estado, ciclos, ramas condicionales, memoria y control de ejecución.	Cuando el agente necesita varios pasos, decisiones, loops, paralelismo, human-in-the-loop o control explícito del flujo.	GitHub - langchain-ai/langgraph : Build resilient agents. · GitHub
LangSmith	Plataforma de observabilidad, trazas, debugging, evaluación y monitoreo de aplicaciones LLM.	Cuando necesitás ver qué hizo el agente, comparar respuestas, evaluar datasets, medir calidad o depurar fallos.	LangSmith docs / Academy: docs.smith.langchain.com intro-to-langsmith/notebooks at main · langchain-ai/intro-to-langsmith · GitHub



Lo esencial de LangChain

ChatOpenAI / ChatAnthropic	Motor LLM	Conecta con el modelo. Punto de entrada a cualquier chain.
ChatPromptTemplate	Plantilla	Define el prompt con variables <code>{variable}</code> . Separa la instrucción del dato. ChatPromptTemplate langchain_core LangChain Reference
StrOutputParser	Parser texto	Convierte el objeto <code>AIMessage</code> en string plano. Opcional — la chain funciona sin él. StrOutputParser langchain_core LangChain Reference
prompt model parser	Chain (LCEL)	Une componentes con <code> </code> . La salida de cada paso entra al siguiente. Es un <code>Runnable</code> . LangChain Expression Language , LCEL Chains and Composition Patterns estelledc/langchain-langgraph-langsmith-tutorial DeepWiki
.invoke() / .batch() / .stream()	Ejecución	<code>invoke</code> = una vez. <code>batch</code> = lista de inputs. <code>stream</code> = token a token. LLM Basics: Invoke, Stream, and Batch estelledc/langchain-langgraph-langsmith-tutorial DeepWiki ←
model.with_structured_output(Schema)	Salida tipada	Structured output - Docs by LangChain
@tool	Define tool	Decora una función Python. El docstring es la descripción que lee el modelo. Tools - Docs by LangChain
model.bind_tools(tools)	Registra tools	Le informa al LLM qué herramientas puede invocar. Sin esto el modelo no las ve. bind_tools langchain_core LangChain Reference

.invoke() / .batch() / .stream()

```
prompt = ChatPromptTemplate.from_template( "Respondé la pregunta: {pregunta}\nUsando este contexto: {contexto}")
```

```
chain.batch([  
    {  
        "pregunta": "¿Qué es RAG?",  
        "contexto": "RAG combina recuperación de documentos con generación."  
    },  
    {  
        "pregunta": "¿Qué son las tools?",  
        "contexto": "Las tools son funciones externas invocables por el modelo."  
    }  
])
```

```
chain.invoke({  
    "pregunta": "¿Qué es RAG?",  
    "contexto": "RAG combina recuperación de documentos con generación."  
})
```

Stream usa la misma estructura de entrada que invoke, pero cambia la forma de entrega. En vez de esperar la respuesta final completa, la chain va devolviendo fragmentos a medida que el modelo genera. Esto es muy útil para interfaces conversacionales

Lo esencial de LangGraph

Concepto	Rol	Para qué sirve
MessagesState	Estado base	Estado predefinido que acumula mensajes. Suficiente para la mayoría de agentes conversacionales. StateGraph and MessagesState langchain-ai/langchain-academy DeepWiki
StateGraph(State)	El grafo	Contenedor del agente. Define qué estado comparten los nodos y cómo fluye la ejecución. StateGraph Fundamentals estelledc/langchain-langgraph-langsmith-tutorial DeepWiki
graph.add_node("nombre", fn)	Agrega nodo	Cada nodo es una función que recibe el estado y retorna estado actualizado. Nodes and Edges langchain-ai/langgraph-101 DeepWiki
graph.add_edge()	Arista fija	Conexión directa entre dos nodos. Siempre va de A a B sin condición. Nodes and Edges langchain-ai/langgraph-101 DeepWiki
add_conditional_edges()	El loop	Decide adónde va el flujo según el estado. Si hay tool_call → tools; si no → END. Acá vive el ciclo. add_conditional_edges langgraph LangChain Reference
ToolNode(tools)	Ejecuta tools en paralelo	Recibe el tool_call del modelo, ejecuta la función y devuelve el resultado al grafo. ToolNode and Tool Execution langchain-ai/langgraph DeepWiki
graph.compile(checkpointer=...)	Construye	Convierte el grafo en un Runnable. Sin checkpointer = sin memoria entre turnos. compile langgraph LangChain Reference
MemorySaver()	Memoria RAM	Checkpoint en memoria para dev/tests. En producción se reemplaza por Redis o Postgres. Agent Memory with Checkpointers langchain-ai/langchain-academy DeepWiki
{"configurable": {"thread_id": "x"}}	Hilo / sesión	Identifica la conversación del usuario. Aísla el estado entre usuarios. Use threads - Docs by LangChain

Lo esencial de LangSmith

Concepto	Rol	Para qué sirve
<code>LANGCHAIN_TRACING_V2=true</code>	Activar trazas	Variable de entorno. Con esto solo, LangSmith registra cada llamada automáticamente. Cero código extra. Trace LangChain applications (Python and JS/TS) - Docs by LangChain
<code>LANGCHAIN_API_KEY</code>	Autenticación	API key del proyecto en LangSmith. Se setea junto con la variable anterior.
<code>LANGCHAIN_PROJECT</code>	Proyecto	Nombre del proyecto donde se agrupan las trazas. Útil para separar dev / staging / prod.
<code>@traceable</code>	Trazar función	Decora cualquier función Python para que aparezca como span en la traza, aunque no use LangChain. Custom instrumentation - Docs by LangChain
Run (traza)	Unidad base	Cada invocación genera un árbol de runs: chain → LLM call → tool call. Con inputs, outputs y latencia.
Dataset + evaluate()	Evaluación	Corre el agente sobre ejemplos y aplica evaluadores automáticamente.
Feedback	Feedback humano	Etiqueta trazas como buenas/malas desde la UI o SDK. Base para mejora del prompt o fine-tuning.

**Veamos el ejemplo y tomemos 30 min
para hacerlo en grupos**

Que significa la salida de esta muestra sintetica?

**Gracias por su atención y
dedicación.**

Recuerden que los grandes retos traen grandes aprendizajes.

¡Nos vemos en la próxima clase!